

Signal Audio Manager

Audio System for Unity

*A flexible, high-performance, data-driven audio solution
designed for fast workflow and professional results*

Version 1.0
October 7, 2025

Contents

Introduction	2
1 Key Features	2
2 Quick Start Guide	2
2.1 Initial Project Setup	2
2.2 Scene Setup	3
3 How to Use	3
3.1 Playing Sounds from the Inspector	3
3.2 Playing Sounds from Code	4
3.3 API Reference	4
4 License	5
4.1 Grant of License	5
4.2 Permitted Uses	5
4.3 Restrictions	5

Introduction

Managing audio in Unity can quickly become complex. Hard-coded references, disorganized management scripts, and inefficient resource loading can slow down development and impact performance.

Signal Audio Manager solves these problems by providing a robust, decoupled framework that is easy to use for beginners and powerful enough for advanced projects.

Built on a foundation of Scriptable Objects and a central event channel, Signal allows you to manage all your game's audio — music, sound effects, UI, and ambient tracks — from a clean, centralized database, with a simple and elegant coding API.

1 Key Features

-  **Data-Driven:** Configure all sounds through a central Scriptable Objects database. No more hard-coded `Resources.Load` paths.
 -  **Decoupled Architecture:** Use the clean Signal API to play sounds from anywhere in your code without needing direct references to the manager.
 -  **Audio Variation:** Automatically add variety to your sound effects. Assign multiple clips to a single ID and set random pitch and volume ranges to make your game sound more alive.
 -  **High Performance:** Uses efficient object pooling for sound effects to avoid garbage collection and runtime overhead.
 -  **Dedicated UI Channel:** UI sounds run in a separate mixer group and use unscaled time, ensuring they play correctly even when the game is paused.
 -  **Ambient Sound System:** Easily create and trigger dynamic ambient soundscapes with crossfade based on player location using simple trigger components.
- Instance Control:** Get a unique ID for looping sounds, allowing you to start and stop specific instances manually (e.g., machine hums, alarms).
-  **Custom Editor Tools:** An easy-to-use custom inspector with a built-in import tool makes adding dozens of audio clips a one-click process.

2 Quick Start Guide

Tip

If you don't want to manually set up your scene, use the **Signal Audio - Setup** folder. It has everything configured, you just need to drag the Managers prefab into your scene!

Follow these steps to get Signal Audio Manager working in your project in minutes.

2.1 Initial Project Setup

(One-time setup)

First, you need to create the central configuration assets for your project.

1. **Create the Configuration File:** In your Project window, right-click and go to: `Create > Signal Audio Manager > Configuration`
Name this new asset `Project_AudioSettings`.
2. **Configure It:** Select the `Project_AudioSettings` asset. In the Inspector, drag your main **Audio Mixer** and your **SFX Prefab** to their respective slots.
3. **Create Ambient Types (Optional):** If you're going to use the ambient system, create your ambient types by right-clicking and going to:
`Create > Signal Audio Manager > Ambient Type`
Create assets like *Forest*, *Cave*, *City*, etc.
4. **Populate the Audio Database:** Select your `Project_AudioSettings` asset again. Now you can add your audio clips:
 - **Import Tool (Recommended):** For each database (Music or SFX/UI), drag a folder containing your audio files to the Source Folder field, set the File Prefix (e.g., `sfx_`) and click **Import**. This will automatically add and group your clips.
 - **Manual:** Click the + in a database list to add a new `AudioEntry`. Give it a unique `AudioID` and add one or more `AudioClips` to its list.

2.2 Scene Setup

Now, in any scene where you want audio, do the following:

1. **Create the Manager GameObject:** Create an empty `GameObject` in your scene. A good name is `_Systems` or `_AudioManager`.
2. **Add Core Components:** Add the `SoundManager` and `AmbientSoundManager` components to this `GameObject`.
3. **Link the Configuration:** This is the most important step. In the `SoundManager` component, find the **Config** field and drag your `Project_AudioSettings` asset to it.
4. **Configure Ambient (Optional):** In the `AmbientSoundManager` component, add entries to the Ambient Configs list. For each entry, link an `AmbientTypeSO` (e.g., `Forest`) and enter the `AudioID` of the music that should play for it.

Information

That's it! The system is now working in your scene.

3 How to Use

3.1 Playing Sounds from the Inspector

For simple event-driven sounds (UI buttons, doors, triggers), use the provided components:

- **PlaySFX / PlayMusic:** Add this component to a `GameObject`. Enter the `AudioID` you want to play. Then call its public `Play()` method from a Unity Event (like a `Button`'s `OnClick()`).
- **AmbientTrigger:** Add this to a `GameObject` with a `Collider` set to "Is Trigger". Assign the Ambient Type you want to activate. When the player enters, the ambient sound will automatically change.

3.2 Playing Sounds from Code

For dynamic sounds, use the static, simple, and clean [Signal] API from any script.

Basic Sound Effects

```
1 // Play a 2D sound (like a UI sound)
2 Signal.PlayUI("button_click");
3
4 // Play a 3D sound at a specific position.
5 // If the "explosion" ID has variation, it will be applied automatically
6 Signal.PlaySFX("explosion", transform.position);
```

Listing 1: Playing sound effects

Music and Ambient

```
1 // Play the main theme with a 2-second crossfade.
2 Signal.PlayMusic("main_theme", 2.0f);
3
4 // Stop the currently playing music with a default fade-out.
5 Signal.StopMusic();
```

Listing 2: Music control

Controlling Looping Instances

```
1 public class Machine : MonoBehaviour
2 {
3     private long machineHumInstanceId = 0;
4
5     public void Activate()
6     {
7         // Start playing a looping sound and store its unique ID.
8         machineHumInstanceId = Signal.PlaySFXLoop("machine_hum",
9             transform);
10    }
11
12    public void Deactivate()
13    {
14        // Use the stored ID to stop this specific sound instance.
15        Signal.StopInstance(machineHumInstanceId);
16        machineHumInstanceId = 0; // Reset the ID
17    }
18 }
```

Listing 3: Managing looping sounds

3.3 API Reference

All methods are static and can be called from `Signal`.

Method	Description
<code>Signal.PlaySFX(string id)</code>	Plays a 2D SFX.
<code>Signal.PlaySFX(string id, Vector3 pos)</code>	Plays a 3D SFX at a world position.
<code>Signal.PlaySFX(string id, Transform target)</code>	Plays a 3D SFX attached to a Transform.
<code>Signal.PlayUI(string id)</code>	Plays a 2D SFX on the dedicated unscaled UI channel.
<code>Signal.PlayMusic(string id, float fade)</code>	Plays a music track with a specified crossfade duration.
<code>Signal.PlaySFXLoop(string id, Transform t)</code>	Returns a long ID. Plays a looping SFX attached to a Transform.
<code>Signal.StopInstance(long id)</code>	Stops a specific looping SFX instance using the ID returned by <code>PlaySFXLoop</code> .
<code>Signal.StopMusic()</code>	Stops the current music with a default fade-out.
<code>Signal.PauseMusic()</code>	Pauses the current music.
<code>Signal.ResumeMusic()</code>	Resumes paused music.

Table 1: Available methods in the Signal API

4 License

This asset is licensed under a **Commercial Asset Use License (EULA - End-User License Agreement)**.

4.1 Grant of License

Subject to the terms and conditions of this EULA, you are granted a non-exclusive, non-transferable license to use this asset in your personal and commercial projects.

4.2 Permitted Uses

- Use the asset in unlimited personal and commercial projects
- Modify the asset to suit your project needs
- Include the asset in shipped games and applications
- Use the asset in both free and paid products

4.3 Restrictions

- You may **not** redistribute, resell, or sublicense the asset as a standalone product
- You may **not** share or transfer your license to third parties
- You may **not** include the asset in asset packs, templates, or tools intended for redistribution
- You may **not** use the asset in products that compete directly with this asset

Warning

This asset is provided "as is" without warranty of any kind, express or implied. The author shall not be liable for any damages arising from the use of this asset.

 Information

For complete license terms and conditions, please refer to the `LICENSE.md` file included with this asset.